

Python, Conda, VS Code 순서대로 설치하고 실행하기

이 문서는 Python을 처음 쓰는 사용자가 위에서부터 순서대로 따라 하면서 Python 코딩 환경을 준비하는 안내문입니다.

Windows 10/11 개인 PC를 기준으로 설명합니다. 이 안내의 단계별 설명은 **Miniconda**를 기준으로 하지만, 처음 시작하는 초보자에게는 **Anaconda** 설치를 권장합니다. Anaconda는 분석에 자주 쓰는 패키지가 미리 들어 있어 더 편합니다(아래 1.2 Anaconda를 선택하는 경우 참고). 어느 것을 설치하든 이후 과정은 동일하게 따라 하면 됩니다.

0. 오늘 만들 환경

이 안내를 끝까지 따라 하면 아래 상태가 됩니다.

- Miniconda 또는 Anaconda가 설치되어 있다.
- VS Code가 설치되어 있다.
- VS Code에 필요한 확장(Python 등)이 설치되어 있다.
- knue-python이라는 conda 가상환경이 만들어져 있다.
- VS Code에서 knue-python 환경을 선택할 수 있다.
- VS Code 터미널 종류(Command Prompt/PowerShell)를 고를 수 있다.
- hello.py 파일을 만들고 실행할 수 있다.
- matplotlib으로 간단한 그래프를 그릴 수 있다.
- AI 코딩 도구에 분석 코드를 요청할 수 있다.
- Git으로 코드를 관리하고 GitHub에 올릴 수 있다.

1. Miniconda 설치하기

Miniconda는 설치가 가볍고, 필요한 패키지만 직접 추가하는 방식입니다. 명령으로 패키지를 설치하는 데 익숙하다면 Miniconda가 깔끔합니다.

완전 초보자라면 Miniconda 대신 Anaconda 설치를 권장합니다. Anaconda에는 NumPy·pandas·matplotlib·Jupyter 같은 분석 패키지가 처음부터 포함되어 있어, 따로 설치하지 않아도 바로 쓸 수 있습니다. 이 경우 1.1을 건너뛰고 **1.2 Anaconda를 선택하는 경우**로 바로 가도 됩니다.

1.1 Miniconda 내려받기

1. 웹 브라우저에서 Miniconda 공식 설치 안내 페이지를 엽니다.
<https://www.anaconda.com/docs/getting-started/miniconda/install>
2. Windows용 64-bit 설치 파일을 내려받습니다.
3. 내려받은 설치 파일을 실행합니다.
4. 설치 대상은 Just Me를 선택합니다.
5. 설치 경로는 기본값을 사용합니다.
6. PATH에 직접 추가하는 옵션은 선택하지 않는 것을 권장합니다.
7. 설치를 완료합니다.
8. Windows 시작 메뉴에서 Anaconda Prompt 또는 Anaconda Prompt (miniconda3)를 엽니다.
9. 아래 명령을 입력합니다.

```
conda --version
python --version
```

10. 버전 정보가 나오면 설치가 된 것입니다.

1.2 Anaconda를 선택하는 경우 (초보자 권장)

Anaconda는 NumPy, pandas, matplotlib, Jupyter 같은 데이터 분석 패키지가 많이 포함된 큰 설치 파일입니다. 패키지를 따로 설치하지 않아도 바로 쓸 수 있어, **처음 배우는 분께 권장합니다.**

Miniconda 대신 Anaconda를 설치하려면 아래 공식 안내에서 Windows용 설치 파일을 내려받아 설치합니다.

<https://www.anaconda.com/docs/getting-started/anaconda/install>

위 페이지에서 다운로드할 때 이메일 입력(가입) 화면이 나오면 **Skip registration(등록 건너뛰기)**을 눌러 계정 없이 받을 수 있습니다.

가입 화면 없이 설치 파일을 바로 받고 싶으면, 아래 공개 아카이브에서 Windows용 64-bit 설치 파일을 직접 내려받아도 됩니다.

https://repo.anaconda.com/archive/Anaconda3-2025.12-2-Windows-x86_64.exe

(아카이브 전체 목록: <https://repo.anaconda.com/archive/> — 더 최신 버전이 있으면 ...-Windows-x86_64.exe 파일을 고르면 됩니다.)

공식 사이트 접속이 느리거나 어려울 때 쓸 수 있는 **대체(미러) 다운로드 링크**입니다.

<https://gofile.me/6VcYy/oNaBh4D64>

대체 링크는 공식 사이트가 아니므로, 받은 파일이 위 공식 아카이브의 설치 파일과 같은지(파일 이름·크기, 가능하면 SHA256 체크섬) 확인하면 더 안전합니다.

설치가 끝나면 Windows 시작 메뉴에서 Anaconda Prompt를 열고 아래 명령으로 확인합니다.

```
conda --version
python --version
```

Miniconda와 Anaconda는 둘 중 하나만 설치하면 됩니다.

2. VS Code 설치하기

1. 웹 브라우저에서 VS Code 공식 다운로드 페이지를 엽니다.

<https://code.visualstudio.com/download>

2. Windows의 User Installer x64를 내려받습니다.
3. 설치 파일을 실행합니다.
4. 설치 옵션에서 아래 항목을 선택하면 편합니다.

- Add to PATH
- Open with Code 관련 항목

5. 설치가 끝나면 VS Code를 실행합니다.

3. VS Code에 필요한 확장 설치하기

확장(Extension)은 VS Code에 기능을 더해 주는 추가 프로그램입니다. 이 안내를 끝까지 따라 하려면 아래 확장을 설치합니다.

설치 방법은 모든 확장이 같습니다.

1. VS Code를 엽니다.
2. 왼쪽 Activity Bar에서 Extensions 아이콘(네모 4개 모양)을 클릭합니다. (단축키 Ctrl+Shift+X)
3. 검색창에 확장 이름을 입력합니다.

4. **제작자(Publisher)를 꼭 확인하고** Install을 누릅니다. 이름이 같은 다른 확장이 있을 수 있습니다. 이 안내에서 사용하는 확장은 아래와 같습니다.

확장 이름	제작자	용도	필요도
Python	Microsoft	Python 실행, 디버깅, 인터프리터 선택	필수
Pylance	Microsoft	빠른 자동완성과 문법 검사	필수
Jupyter	Microsoft	노트북(.ipynb) 실행, 셀 단위로 그래프 확인	권장
GitHub Copilot	GitHub	AI 코드 제안	선택
GitHub Copilot Chat	GitHub	대화형으로 코드 요청	선택
GitLens	GitKraken	Git 변경 이력을 보기 쉽게 표시	선택
Korean Language Pack	Microsoft	VS Code 메뉴를 한국어로 표시	선택

- Python 확장을 설치하면 Pylance가 함께 설치되는 경우가 많습니다.
- 그래프나 노트북 작업을 할 예정이면 Jupyter를 설치합니다.
- AI 코딩 도구(Copilot 등)는 **11장**에서 다시 설명합니다.
- Git 관련 도구는 **15장**에서 다시 설명합니다.

설치가 끝나면 VS Code를 한 번 다시 시작합니다.

4. Python 작업 폴더 만들기

1. Python 코드를 저장할 폴더를 만듭니다.

예:

C:\Mydata\python_practice\

2. VS Code를 엽니다.
3. 메뉴에서 File > Open Folder...를 선택합니다.
4. 방금 만든 폴더를 엽니다.
5. VS Code가 이 폴더를 신뢰할지 물으면 Yes, I trust the authors를 선택합니다.

이어서 **자동 저장(Auto Save)**을 켜는 것을 권장합니다. 자동 저장을 켜면 코드를 고친 뒤 따로 저장하지 않아도 되어, 저장을 깜빡해 생기는 실수를 줄일 수 있습니다.

6. 메뉴에서 File을 엽니다.

7. Auto Save 항목을 클릭해 체크 표시가 커지도록 합니다. 체크가 켜져 있으면 자동 저장이 동작합니다.

더 세밀하게 설정하고 싶으면 아래처럼 합니다.

- Ctrl+, (쉼표)를 눌러 설정(Settings)을 엽니다.
- 검색창에 Auto Save를 입력합니다.
- Files: Auto Save 값을 afterDelay(잠시 후 자동 저장)로 선택합니다.

자동 저장을 켜 두면 이후 단계에서 “파일을 저장합니다”라고 나오는 부분을 따로 하지 않아도 됩니다. 다만 새로 만든 파일은 이름을 정해 **한 번** 저장해야 그때부터 자동 저장이 시작됩니다.

5. conda 가상환경 만들기

가상환경은 프로젝트마다 Python과 패키지를 분리해서 관리하는 작업 공간입니다. 여기서는 knue-python이라는 환경을 만듭니다.

conda 명령은 두 가지 방법 중 편한 곳에서 입력하면 됩니다.

- **방법 1: Anaconda Prompt** — Windows 시작 메뉴에서 Anaconda Prompt 또는 Anaconda Prompt (miniconda3)를 엽니다. conda 설정이 끝난 상태로 열려서 가장 확실합니다.
- **방법 2: VS Code 터미널** — VS Code에서 작업 폴더를 연 뒤 메뉴 Terminal > New Terminal로 터미널을 열어 같은 명령을 입력해도 됩니다. 굳이 Anaconda Prompt를 따로 열지 않아도 됩니다.

VS Code 터미널에서 conda가 인식되지 않거나 환경 이름이 프롬프트에 보이지 않으면, 터미널 종류 때문일 수 있습니다. 이럴 때는 **7장(VS Code 터미널 선택)**을 참고해 터미널을 Command Prompt로 바꾸거나 conda init을 한 번 실행합니다.

아래 순서대로 진행합니다.

1. 위 방법 중 하나로 터미널(또는 Anaconda Prompt)을 엽니다.
2. 아래 명령을 입력합니다.

```
conda create -n knue-python python=3.12
```

3. 설치할 패키지 목록이 나오면 y를 입력하고 Enter를 누릅니다.
4. 환경을 활성화합니다.

```
conda activate knue-python
```

5. Python 버전을 확인합니다.

```
python --version
```

6. 기본 패키지를 설치합니다.

```
conda install numpy pandas matplotlib
```

7. 설치할지 물으면 y를 입력하고 Enter를 누릅니다.

6. VS Code에서 conda 환경 선택하기

1. VS Code에서 앞에서 만든 Python 작업 폴더를 열어 둡니다.
2. Ctrl+Shift+P를 누릅니다.
3. Python: Select Interpreter를 검색해서 실행합니다.
4. 목록에서 knue-python이 들어간 Python 환경을 선택합니다.
5. VS Code 오른쪽 아래 상태 표시줄에 knue-python 또는 선택한 Python 버전이 보이는지 확인합니다.
6. VS Code에서 새 터미널을 엽니다.

메뉴: Terminal > New Terminal

7. 터미널에서 아래 명령을 입력합니다.

```
python --version
```

```
conda info --envs
```

8. knue-python 환경에 별표 *가 있거나, VS Code가 선택한 Python 경로가 knue-python과 연결되어 있으면 됩니다.

터미널에서 conda activate가 잘 안 되거나 환경 이름이 프롬프트에 보이지 않으면, 사용 중인 터미널 종류 때문일 수 있습니다. 다음 **7장**에서 터미널 선택 방법과 conda 설정을 확인하세요.

7. VS Code 터미널 선택하기 (Command Prompt와 PowerShell)

VS Code 안에서도 명령을 입력하는 터미널을 사용합니다. Windows에서 VS Code 터미널은 여러 종류 중 하나를 고를 수 있는데, conda를 쓸 때는 어떤 터미널을 쓰는지 중요합니다.

7.1 터미널 선택 메뉴 사용하기

1. VS Code 메뉴에서 Terminal > New Terminal을 선택해 터미널 패널을 엽니다.
2. 터미널 패널 오른쪽 위를 보면 새 터미널을 여는 + 버튼과, 그 옆에 아래 방향 화살표 모양의 드롭다운 버튼이 있습니다.
3. 그 드롭다운 버튼을 누르면 사용할 수 있는 터미널 종류 목록이 나옵니다.
 - PowerShell
 - Command Prompt
 - Git Bash (Git을 설치한 경우에만 보입니다)
4. 원하는 터미널 종류를 선택하면 그 종류로 새 터미널이 열립니다.

기본으로 열리는 터미널 종류를 바꾸려면 아래처럼 합니다.

1. Ctrl+Shift+P를 누릅니다.
2. Terminal: Select Default Profile을 검색해서 실행합니다.
3. 목록에서 기본으로 사용할 터미널(Command Prompt 또는 PowerShell)을 선택합니다.
4. 다음부터 Terminal > New Terminal로 새 터미널을 열면 선택한 종류로 열립니다.

7.2 Command Prompt와 PowerShell의 장단점

Windows의 VS Code 터미널에서 자주 쓰는 두 가지는 Command Prompt와 PowerShell입니다. 둘은 아래와 같은 차이가 있습니다.

항목	Command Prompt (cmd)	PowerShell
프롬프트 모양	C:\Users\이름>	PS C:\Users\이름>
성격	오래된 Windows 기본 명령창	더 최신이고 기능이 많은 셸
명령 기능	기본 명령 위주로 단순함	변수, 스크립트, 객체 처리 등 풍부함
명령 구분 문자	& 등으로 단순함	세미콜론 ; 등으로 다양함
conda 사용	활성화가 추가 설정 없이 되는 경우가 많음	처음에는 conda init 설정이 필요할 수 있음

Command Prompt (cmd)

- 장점: 화면이 단순하고, conda 환경 활성화가 추가 설정 없이 바로 되는 경우가 많습니다. 명령이 짧아 처음 배우기 쉽습니다.
- 단점: 변수·스크립트 같은 고급 기능이 부족하고, 최신 도구 안내가 PowerShell 기준으로 적혀 있는 경우가 많습니다.

PowerShell

- 장점: Windows의 기본 터미널이고 기능이 풍부합니다. 대부분의 최신 문서와 도구 설명이 PowerShell을 기준으로 하며, 자동완성과 스크립트 작성에 유리합니다.
- 단점: conda를 처음 쓸 때 conda init powershell 설정이 필요할 수 있고, 환경 이름이 프롬프트에 바로 보이지 않는 경우가 있습니다.

둘 중 무엇을 쓸지는 **사용자가 정하면 됩니다**. Python과 conda만 단순히 쓰고 싶으면 Command Prompt가 편하고, 다양한 명령과 최신 도구 안내를 그대로 따라 하고 싶으면 PowerShell이 편합니다. 어느 쪽을 골라도 이 안내의 명령은 모두 동작합니다. PowerShell에서 conda가 바로 잡히지 않을 때의 설정은 다음 7.3을 참고하세요.

7.3 conda가 터미널에서 안 잡힐 때

VS Code 터미널에서 `conda activate knue-python`을 입력했는데 환경 이름이 프롬프트 앞에 표시되지 않거나, conda를 찾을 수 없다는 메시지가 나오면 아래 중 하나로 해결합니다.

1. **PowerShell을 쓰는 경우**: 한 번만 아래 설정을 합니다. 시작 메뉴에서 Anaconda Prompt를 열고 아래 명령을 입력한 뒤 VS Code를 다시 시작합니다.

```
conda init powershell
```

2. **간단하게 해결하고 싶은 경우**: 터미널 종류를 Command Prompt로 바꿔서 다시 시도합니다. (7.1 참고) Command Prompt에서는 추가 설정 없이 활성화되는 경우가 많습니다.
3. 그래도 안 되면 VS Code를 완전히 닫았다가 다시 열고, Python: Select Interpreter로 knue-python을 다시 선택합니다.

참고: Windows 시작 메뉴의 Anaconda Prompt는 conda 설정이 이미 끝난 상태로 열리는 전용 Command Prompt입니다. VS Code 터미널에서 conda가 잘 안 잡힐 때는 Anaconda Prompt에서 먼저 명령이 되는지 확인하면 원인을 좁히기 쉽습니다.

8. 첫 Python 파일 만들기

1. VS Code 왼쪽 Explorer에서 새 파일을 만듭니다.
2. 파일 이름을 `hello.py`로 저장합니다.
3. 아래 코드를 그대로 입력합니다.

```
print("Hello, Python!")
```

```
name = "KNUE"
```

```
print(f"Hello, {name} thesis project.")
```

4. 파일을 저장합니다.
5. 터미널에서 아래 명령을 입력합니다.

```
python hello.py
```

6. 터미널에 아래처럼 나오면 성공입니다.

```
Hello, Python!
```

```
Hello, KNUE thesis project.
```

9. VS Code 실행 버튼으로 실행하기

터미널 명령 대신 VS Code 실행 버튼으로도 Python 파일을 실행할 수 있습니다.

1. `hello.py` 파일을 엽니다.
2. 오른쪽 위의 실행 버튼을 누릅니다.
3. 터미널에 실행 결과가 나오는지 확인합니다.

실행 결과가 이상하면 먼저 VS Code에서 선택된 Python 인터프리터가 knue-python인지 확인합니다.

10. 그래프 예제 실행하기

이번에는 matplotlib으로 간단한 그래프를 그립니다.

1. VS Code에서 새 파일을 만듭니다.
2. 파일 이름을 plot_example.py로 저장합니다.
3. 아래 코드를 입력합니다.

```
import matplotlib.pyplot as plt
```

```
years = [2022, 2023, 2024, 2025, 2026]  
values = [3, 5, 6, 8, 11]
```

```
plt.plot(years, values, marker="o")  
plt.xlabel("Year")  
plt.ylabel("Value")  
plt.title("Example Plot")  
plt.grid(True)  
plt.show()
```

4. 터미널에서 실행합니다.

```
python plot_example.py
```

5. 그래프 창이 열리면 패키지 설치와 Python 실행이 된 것입니다.

11. AI 코딩 도구로 Python 코드 요청하기

논문을 작성하다 보면 설문 자료, 실험 결과, 인터뷰 코딩 자료를 Python으로 분석해야 할 때가 있습니다. 이때 AI 코딩 도구에게 분석 코드를 요청하면 초안 코드를 빠르게 만들 수 있습니다.

대표적인 도구로 **GitHub Copilot**, **Claude Code**, **GPT Codex**가 있습니다. 도구마다 화면은 조금씩 다르지만, 좋은 프롬프트(요청문)를 작성하는 방법은 거의 같습니다. 아래에서는 먼저 GitHub Copilot 설치를 예로 들고, 이어서 세 도구에 코드를 요청하는 방법과 프롬프트 예시를 설명합니다.

11.1 GitHub Copilot 설치하기

1. VS Code를 엽니다.
2. 왼쪽 Activity Bar에서 Extensions 아이콘을 클릭합니다.
3. 검색창에 GitHub Copilot을 입력합니다.
4. 제작자가 GitHub인 GitHub Copilot 또는 GitHub Copilot Chat을 설치합니다.
5. VS Code 오른쪽 아래 또는 Accounts 메뉴에서 GitHub 계정으로 로그인합니다.
6. Copilot Chat 창이 열리는지 확인합니다.

GitHub Copilot은 계정이나 요금제에 따라 사용할 수 있는 기능과 횟수가 다를 수 있습니다. 학교나 개인 계정의 사용 가능 여부를 먼저 확인합니다.

11.2 GitHub Copilot, Claude Code, GPT Codex에 코드 요청하기

세 도구 모두 “무엇을 만들고 싶은지”를 한국어(또는 영어)로 적어 보내면 Python 코드를 만들어 줍니다. 보내는 위치와 방식만 조금 다릅니다.

도구	어디서 요청하나	특징
GitHub Copilot	VS Code 안의 Copilot Chat 창, 또는 코드 편집 중 인라인 제안	VS Code에 통합되어 지금 편집 중인 파일을 바로 참고함
Claude Code	터미널에서 claude 명령으로 실행하거나 VS Code 확장으로 사용	폴더 전체를 읽고 파일 생성·수정·실행까지 대신 해 줌
GPT Codex	ChatGPT의 Codex 기능 또는 VS Code 확장	대화하며 코드를 단계적으로 만들고 다듬기 좋음

Claude Code의 설치(네이티브 설치·Git for Windows 필수)와 로그인, VS Code PowerShell에서 실행하는 방법은 [6-CLAUDE_CODE_SETUP.md](#)에 따로 정리해 두었습니다.

요청하는 기본 흐름은 같습니다.

1. 채팅 창이나 입력란을 엽니다.
 - GitHub Copilot: VS Code 오른쪽의 Copilot Chat 창
 - Claude Code: VS Code 터미널에서 claude를 입력한 뒤 대화
 - GPT Codex: ChatGPT 화면 또는 VS Code 확장 창
2. 만들고 싶은 코드를 자연어로 적습니다. (아래 11.3, 11.4의 작성 요령을 참고합니다.)
3. 만들어진 코드를 파일로 저장합니다.
4. 터미널에서 직접 실행해 결과를 확인합니다.

공통으로 지키면 좋은 점:

- 한 번에 너무 많은 것을 요청하지 말고, 작은 단위로 나눠서 요청합니다.
- 입력 데이터의 형태(열 이름, 예시 값)와 원하는 출력(파일 이름, 그림 형태)을 구체적으로 적습니다.
- 사용할 패키지, Python 버전, 가상환경 이름을 함께 알려 줍니다.
- 만들어진 코드는 그대로 쓰지 말고, 직접 읽고 실행해 결과를 확인합니다.
- 개인정보가 든 원자료는 그대로 붙여 넣지 말고, 익명화한 예시나 열 구조만 공유합니다.

11.3 AI에게 요청할 때 포함할 내용

AI에게 코드를 요청할 때는 아래 정보를 함께 주면 좋습니다.

1. 분석 목적
2. 데이터 파일 이름
3. 데이터 열 이름
4. 원하는 출력물
5. 사용할 Python 패키지
6. 초보자용 주석이 필요한지 여부

좋은 요청 예시는 아래와 같습니다.

나는 논문 작성을 위해 인터뷰 코딩 자료로 간단한 ENA 스타일 네트워크 그림을 그리고 싶습니다.

조건:

- Python 초보자도 이해할 수 있게 주석을 달아 주세요.
- pandas와 matplotlib만 사용해 주세요.
- 예시 데이터는 코드 안에 직접 넣어 주세요.
- 각 발화에는 여러 개의 코드가 세미콜론으로 구분되어 있다고 가정해 주세요.
- 같은 발화 안에 함께 나온 코드 쌍을 세어서 네트워크 선 굵기로 표현해 주세요.
- 결과 그림은 ena_example.png로 저장하고 화면에도 보여 주세요.
- 실행 파일 이름은 ena_example.py로 할 예정입니다.

AI가 만든 코드는 바로 논문에 쓰기 전에 반드시 직접 읽고 실행해 봅니다. 분석 방법, 변수명, 데이터 전처리 방식이 내 연구 설계와 맞는지도 확인해야 합니다.

11.4 과학교육 연구용 Python 프로젝트 프롬프트 예시

실제 연구에서는 코드 한 개보다, **폴더 구조가 정리된 작은 프로젝트**를 통째로 요청하는 것이 편합니다. 특히 모듈(패키지) 버전이 서로 충돌해 생기는 오류를 피하려면, **이 프로젝트 전용 가상환경**을 따로 만들어 쓰는 것이 좋습니다.

아래는 데이터 폴더(data), 코드 폴더(src), 결과 폴더(output)를 구분하고, 가상환경과 필요한 모듈 설치까지 포함하도록 요청하는 프롬프트 예시입니다. GitHub Copilot, Claude Code, GPT Codex 어디에 붙여 넣어도 됩니다.

나는 과학교육 연구 자료를 분석하는 작은 Python 프로젝트를 만들고 싶습니다.
초보자도 따라 할 수 있게 폴더 구조와 실행 순서를 함께 알려 주세요.

[환경]

- Windows에서 conda 가상환경을 사용합니다.
- 모듈(패키지) 버전이 충돌하는 오류를 피하기 위해, 이 프로젝트 전용 가상환경을 새로 만들고 싶습니다.
- 가상환경 이름: sci-edu-analysis
- Python 버전: 3.12

[폴더 구조] 아래처럼 폴더를 나눠 주세요.

- data/ : 원본 데이터(csv 등)를 두는 폴더
- src/ : 실행할 Python 코드를 두는 폴더
- output/ : 그림, 표, 결과 파일을 저장하는 폴더

[데이터]

- 파일 위치: data/survey.csv
- 열 이름:
 - student_id : 학생 구분 번호
 - pre_score : 사전 검사 점수
 - post_score : 사후 검사 점수
 - group : 실험집단(experiment) 또는 비교집단(control)

[분석 목적]

- 집단별로 사전·사후 점수의 평균과 표준편차를 계산해 주세요.
- 사전 대비 사후 점수의 향상도를 집단별로 비교해 주세요.
- 결과를 막대그래프로 그려 output/score_comparison.png로 저장해 주세요.
- 요약 표는 output/summary.csv로 저장해 주세요.

[요청 사항]

1. 먼저 conda 가상환경을 만들고 활성화하는 명령을 순서대로 알려 주세요.
2. 이 분석에 필요한 모듈(pandas, matplotlib 등)을 설치하는 명령도 알려 주세요.
3. 코드는 src/analyze.py 한 파일에 작성하고, data/와 output/ 경로를 코드 위쪽에서 변수로 정리해 주세요.
4. 데이터 파일이 없을 때를 대비해, 같은 열 구조의 예시 data/survey.csv를 만드는 코드도 함께 주세요.
5. Python 초보자가 이해할 수 있게 주요 줄에 한국어 주석을 달아 주세요.
6. 마지막에 전체 실행 순서를 1, 2, 3 단계로 정리해 주세요.

이렇게 요청하면 보통 아래와 같은 순서의 안내와 코드를 받게 됩니다.

1) 프로젝트 전용 가상환경 만들기

```
conda create -n sci-edu-analysis python=3.12
```

2) 가상환경 활성화

```
conda activate sci-edu-analysis
```

3) 필요한 모듈 설치

```
conda install pandas matplotlib
```

4) 코드 실행 (프로젝트 폴더에서)

```
python src/analyze.py
```

폴더를 나누면 좋은 점:

- data와 output을 분리하면 원본 데이터를 실수로 덮어쓰지 않습니다.
- src에 코드만 모으면 어떤 파일을 실행해야 하는지 한눈에 보입니다.
- 프로젝트 전용 가상환경을 쓰면, 다른 프로젝트의 모듈 버전과 충돌하는 오류를 막을 수 있습니다.

모듈 버전 충돌 오류(예: 한 패키지는 numpy 최신 버전을, 다른 패키지는 옛 버전을 요구하는 경우)는 한 환경에 여러 프로젝트의 패키지를 섞어 설치할 때 자주 생깁니다. 프로젝트마다 가상환경을 따로 만들면 이런 문제를 크게 줄일 수 있습니다.

12. ENA 스타일 네트워크 예제 실행하기

ENA는 학습자 발화나 글에서 여러 코드가 어떻게 함께 나타나는지 살펴볼 때 사용할 수 있는 네트워크 기반 분석 방법입니다. 아래 예시는 정식 ENA 통계 분석 전체가 아니라, 초보자가 Python 실행 흐름을 익히기 위한 **ENA 스타일 코드 공동출현 네트워크** 그림입니다.

12.1 예제 파일 만들기

1. VS Code에서 새 파일을 만듭니다.
2. 파일 이름을 ena_example.py로 저장합니다.
3. 아래 코드를 붙여 넣습니다.

```
from itertools import combinations
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

예시 데이터입니다.

실제 논문 자료에서는 speaker, turn, codes 열을 가진 CSV 파일로 바꿔 사용할 수 있습니다.

```
data = [
    {"speaker": "S1", "turn": 1, "codes": "claim;evidence"},
    {"speaker": "S1", "turn": 2, "codes": "claim;reasoning"},
    {"speaker": "S2", "turn": 3, "codes": "question;evidence"},
    {"speaker": "S2", "turn": 4, "codes": "reasoning;reflection"},
    {"speaker": "S3", "turn": 5, "codes": "claim;evidence;reasoning"},
    {"speaker": "S3", "turn": 6, "codes": "question;reflection"},
    {"speaker": "S1", "turn": 7, "codes": "evidence;reasoning"},
    {"speaker": "S2", "turn": 8, "codes": "claim;reflection"},
]
```

```
df = pd.DataFrame(data)
```

네트워크에 표시할 코드 목록과 위치를 정합니다.

```
positions = {
    "claim": (0.0, 1.0),
    "evidence": (1.0, 0.3),
    "reasoning": (0.6, -0.9),
}
```

```

    "question": (-0.6, -0.9),
    "reflection": (-1.0, 0.3),
}

# 같은 발화 안에 함께 나온 코드 쌍을 셉니다.
edge_counts = {}

for codes_text in df["codes"]:
    codes = [code.strip() for code in codes_text.split(";")]
    codes = sorted(set(codes))

    for code_a, code_b in combinations(codes, 2):
        pair = tuple(sorted([code_a, code_b]))
        edge_counts[pair] = edge_counts.get(pair, 0) + 1

# 그림을 그립니다.
fig, ax = plt.subplots(figsize=(7, 6))

max_count = max(edge_counts.values())

for (code_a, code_b), count in edge_counts.items():
    x1, y1 = positions[code_a]
    x2, y2 = positions[code_b]
    line_width = 1 + 4 * (count / max_count)

    ax.plot(
        [x1, x2],
        [y1, y2],
        linewidth=line_width,
        color="steelblue",
        alpha=0.65,
    )

    mid_x = (x1 + x2) / 2
    mid_y = (y1 + y2) / 2
    ax.text(mid_x, mid_y, str(count), fontsize=9, color="black")

for code, (x, y) in positions.items():
    ax.scatter(x, y, s=1200, color="white", edgecolor="black", linewidth=1.5, zorder=3)
    ax.text(x, y, code, ha="center", va="center", fontsize=10, zorder=4)

ax.set_title("ENA Style Code Co-occurrence Network")
ax.set_aspect("equal")
ax.axis("off")

plt.tight_layout()
plt.savefig("ena_example.png", dpi=300)
plt.show()

```

12.2 예제 실행하기

1. VS Code에서 ena_example.py를 저장합니다.
2. 터미널에서 knue-python 환경을 활성화합니다.

```
conda activate knue-python
```

3. 필요한 패키지가 설치되어 있는지 확인합니다.

```
conda install pandas matplotlib
```

4. 코드를 실행합니다.

```
python ena_example.py
```

5. 그래프 창이 열리고, 작업 폴더에 ena_example.png 파일이 생기면 성공입니다.

12.3 내 데이터로 바꿀 때

실제 논문 자료를 사용할 때는 AI에게 아래처럼 다시 요청할 수 있습니다.

방금 만든 ENA 스타일 네트워크 코드를 내 CSV 파일을 읽는 방식으로 바꿔 주세요.

파일 이름: interview_codes.csv

열 이름:

- speaker: 참여자 이름
- turn: 발화 번호
- codes: 세미콜론으로 구분된 코딩 결과

요청:

- pandas로 CSV를 읽어 주세요.
- codes 열에서 코드 쌍의 공동출현 빈도를 계산해 주세요.
- 빈도가 높은 연결일수록 선을 굵게 그려 주세요.
- 결과를 ena_interview_network.png로 저장해 주세요.
- 초보자가 수정할 부분에는 주석을 달아 주세요.

AI가 만든 코드를 실행할 때는 파일 이름, 열 이름, 저장 위치가 실제 자료와 맞는지 먼저 확인합니다. 연구 자료에 개인정보가 들어 있다면 AI 서비스에 원자료를 그대로 붙여 넣지 말고, 익명화한 예시 데이터나 열 구조만 공유합니다.

13. 추가 패키지 설치하기

나중에 필요한 패키지는 현재 환경을 활성화한 뒤 설치합니다.

```
conda activate knue-python
```

```
conda install 패키지이름
```

예:

```
conda install scipy seaborn openpyxl
```

conda에 없는 패키지는 pip로 설치할 수 있습니다.

```
pip install 패키지이름
```

한 프로젝트 안에서는 가능하면 같은 conda 환경에 패키지를 설치합니다.

14. conda 명령어 정리

conda 명령은 크게 **환경 관리**, **패키지 관리**, **정보 확인**으로 나눌 수 있습니다. 자주 쓰는 명령을 종류별로 정리했습니다. 명령은 Anaconda Prompt나 conda가 설정된 VS Code 터미널에서 입력합니다.

14.1 환경 관리

작업	명령
환경 목록 보기	conda info --envs 또는 conda env list
새 환경 만들기	conda create -n 환경이름 python=3.12
환경 활성화	conda activate 환경이름
환경 비활성화	conda deactivate
환경 복제	conda create -n 새이름 --clone 기존이름
환경 삭제	conda remove -n 환경이름 --all

14.2 패키지 관리

작업	명령
패키지 설치	conda install 패키지이름
여러 패키지 한 번에 설치	conda install numpy pandas matplotlib
특정 버전 설치	conda install numpy=1.26
패키지 업데이트	conda update 패키지이름
패키지 삭제	conda remove 패키지이름
설치된 패키지 보기	conda list
conda에 없는 패키지 설치	pip install 패키지이름

14.3 정보 확인과 관리

작업	명령
conda 버전 확인	conda --version
Python 버전 확인	python --version
conda 전체 정보 보기	conda info
conda 자체 업데이트	conda update conda
캐시 정리	conda clean --all

14.4 환경 내보내기와 재현

다른 컴퓨터에서 같은 환경을 다시 만들고 싶을 때는, 환경 내용을 파일로 내보냈다가 그 파일로 환경을 다시 만들 수 있습니다.

현재 환경을 파일로 내보내기 (환경을 활성화한 상태에서 실행)

```
conda env export > environment.yml
```

내보낸 파일로 환경 다시 만들기

```
conda env create -f environment.yml
```

14.5 명령을 쓸 때 팁

- 환경 이름은 영어, 숫자, 하이픈 정도로 단순하게 짓는 것이 좋습니다.
- 패키지를 설치하기 전에 conda activate로 원하는 환경을 먼저 활성화합니다.
- 설치할지 물으면 y를 입력하고 Enter를 누릅니다. 묻지 않고 바로 설치하려면 명령 끝에 -y를 붙입니다.
예: conda install numpy -y
- conda와 pip을 함께 쓸 때는 되도록 conda로 먼저 설치하고, conda에 없는 것만 pip으로 설치합니다.

15. Git과 GitHub로 연구 코드 관리하기

분석 코드가 늘어나면, 코드를 **버전별로 기록하고 안전하게 보관**하는 도구가 필요합니다. 이때 쓰는 것이 Git과 GitHub입니다.

- **Git**: 내 컴퓨터에서 코드의 변경 이력을 관리하는 프로그램입니다.
- **GitHub**: Git으로 관리한 코드를 인터넷에 저장하고 공유하는 웹 서비스입니다.

15.1 Git과 GitHub를 쓰면 좋은 점

- **변경 이력 관리**: 코드를 언제 어떻게 바꿨는지 기록이 남습니다. 문제가 생기면 이전 상태로 되돌릴 수 있습니다.
- **안전한 백업**: GitHub에 올려 두면 컴퓨터가 고장 나도 코드를 잃지 않습니다.
- **버전 비교**: 두 시점의 코드 차이를 한눈에 볼 수 있습니다.
- **협업**: 여러 사람이 같은 코드를 나눠 작업하고 합칠 수 있습니다.
- **논문 재현성**: 결과를 만든 분석 코드의 특정 버전을 그대로 보존할 수 있어, 나중에 같은 결과를 재현하기 쉽습니다.
- **공개와 인용**: 코드를 공개 저장소에 올리고, 필요하면 버전을 지정해 논문에서 인용할 수 있습니다.

15.2 Windows에 Git 설치하기

1. 웹 브라우저에서 Git 공식 다운로드 페이지를 엽니다.
`https://git-scm.com/download/win`
2. 64-bit Windows Setup 파일을 내려받아 실행합니다.
3. 설치 중 대부분은 기본값으로 두면 됩니다. 다만 아래 화면이 나오면 다음을 권장합니다.
 - 기본 편집기 선택: Use Visual Studio Code as Git's default editor (VS Code를 이미 설치했다면)
 - 기본 브랜치 이름: Override the default branch name for new repositories에서 main 입력
 - PATH 설정: Git from the command line and also from 3rd-party software
4. 설치를 완료합니다.
5. 새 터미널(VS Code 터미널, Command Prompt, 또는 Git Bash)을 열고 아래로 확인합니다.

```
git --version
```

버전 정보가 나오면 설치가 된 것입니다.

15.3 Git 최초 설정 (이름과 이메일)

커밋(저장 기록)에 남길 이름과 이메일을 한 번만 설정합니다. GitHub 가입 이메일과 같게 하는 것을 권장합니다.

```
git config --global user.name "Hong Gildong"
git config --global user.email "you@example.com"
```

새로 만드는 저장소의 기본 브랜치 이름을 main으로 둡니다.

```
git config --global init.defaultBranch main
```

설정이 잘 되었는지 확인합니다.

```
git config --global --list
```

15.4 GitHub 계정 만들기

1. 웹 브라우저에서 <https://github.com> 에 접속합니다.
2. Sign up을 눌러 이메일, 비밀번호, 사용자 이름을 입력합니다.
3. 이메일로 온 인증 코드를 입력해 계정을 활성화합니다.
4. 보안을 위해 2단계 인증(2FA) 설정을 권장합니다.

15.5 새 저장소 만들고 코드 올리기

방법은 두 가지입니다. 처음에는 **방법 A**가 더 쉽습니다.

방법 A: GitHub에서 먼저 저장소를 만들고 내려받기(clone)

1. GitHub에 로그인한 뒤 오른쪽 위 + → New repository를 누릅니다.
2. 저장소 이름을 정합니다. 예: my-research-code
3. 공개 범위를 고릅니다. 연구 자료가 들어가면 Private(비공개)를 권장합니다.
4. Add a README file을 체크하고 Create repository를 누릅니다.
5. 저장소 화면에서 초록색 Code 버튼을 누르고 HTTPS 주소를 복사합니다.
6. VS Code 터미널에서 코드를 내려받습니다.

```
git clone https://github.com/사용자이름/my-research-code.git
```

7. 내려받은 폴더 안에서 파일을 만들거나 수정한 뒤, 아래 순서로 올립니다.

```
git add .
git commit -m "분석 코드 추가"
git push
```

방법 B: 이미 있는 내 폴더를 저장소로 만들어 올리기

작업 폴더 안에서 아래를 순서대로 입력합니다.

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/사용자이름/my-research-code.git
git push -u origin main
```

처음 push할 때 GitHub 로그인 창이 뜨면 브라우저로 인증합니다.

15.6 자주 쓰는 Git 명령 정리

작업	명령
현재 상태 보기	git status
변경 파일을 기록 대상에 추가	git add 파일이름 또는 git add .
변경 내용 저장(커밋)	git commit -m "설명"
변경 이력 보기	git log --oneline
GitHub에 올리기	git push
GitHub 변경 내려받기	git pull
저장소 내려받기	git clone 주소
두 시점의 차이 보기	git diff
새 브랜치 만들고 이동	git switch -c 브랜치이름

작업	명령
브랜치 이동	git switch 브랜치이름

commit 메시지는 “무엇을 왜 바꿨는지”를 짧고 분명하게 적습니다.

15.7 VS Code에서 버튼으로 Git 사용하기

명령어가 익숙하지 않으면 VS Code 화면의 버튼으로도 같은 일을 할 수 있습니다.

1. 왼쪽 Activity Bar에서 Source Control 아이콘(가지처럼 갈라진 모양)을 클릭합니다.
2. 변경된 파일 목록이 보입니다. 파일 옆 +를 눌러 기록 대상에 추가(stage)합니다.
3. 위쪽 입력란에 변경 설명을 적고 체크(□) 버튼을 눌러 커밋합니다.
4. ... 메뉴 또는 아래쪽 동기화 버튼으로 Push(올리기), Pull(내려받기)을 합니다.

처음에는 명령어와 버튼 중 편한 방법을 쓰면 됩니다.

15.8 .gitignore로 올리지 않을 파일 지정하기

연구 프로젝트에서는 원본 데이터, 결과 파일, 캐시 파일을 GitHub에 올리지 않는 편이 좋을 때가 많습니다(용량이 크거나 개인정보가 들어 있을 수 있기 때문입니다).

프로젝트 루트에 .gitignore 파일을 만들고 올리지 않을 항목을 적습니다.

```
# Python 캐시
__pycache__/
*.pyc
```

```
# 가상환경
.venv/
env/
```

```
# 데이터와 결과 (필요에 따라)
data/
output/
```

```
# OS / 편집기 파일
.DS_Store
.vscode/
```

주의: 개인정보가 든 데이터는 반드시 제외하거나 익명화합니다. 한 번 GitHub에 올라간 파일은 변경 이력에 계속 남기 때문에, 민감한 정보는 처음부터 올리지 않는 것이 안전합니다.

16. 자주 나는 문제와 해결

conda 명령을 찾을 수 없을 때

일반 PowerShell이 아니라 Anaconda Prompt 또는 Anaconda Prompt (miniconda3)를 열어 확인합니다.

```
conda --version
```

VS Code 터미널에서 conda가 바로 잡히지 않으면 VS Code를 완전히 닫았다가 다시 열고, Python 인터프리터를 다시 선택합니다. 터미널 종류 문제일 수 있으니 7장도 참고합니다.

VS Code가 다른 Python을 사용할 때

VS Code가 Windows에 원래 설치된 Python이나 Microsoft Store Python을 선택했을 수 있습니다.

1. Ctrl+Shift+P를 누릅니다.
2. Python: Select Interpreter를 실행합니다.
3. knue-python 환경을 다시 선택합니다.
4. 터미널을 새로 열고 `python --version`을 확인합니다.

패키지를 설치했는데 import가 안 될 때

패키지를 설치한 환경과 VS Code에서 선택한 환경이 다를 수 있습니다.

1. VS Code에서 선택된 인터프리터를 확인합니다.
2. 터미널에서 현재 환경을 확인합니다.

```
conda info --envs
```

3. 필요한 환경을 활성화한 뒤 다시 설치합니다.

```
conda activate knue-python
```

```
conda install numpy pandas matplotlib
```

모듈 버전이 충돌한다는 오류가 날 때

한 환경에 여러 프로젝트의 패키지를 섞어 설치하면, 패키지들이 요구하는 버전이 서로 달라 충돌이 날 수 있습니다.

1. 프로젝트마다 가상환경을 따로 만듭니다. (5장, 11.4 참고)
2. 새 환경을 만들고 필요한 패키지만 다시 설치합니다.

```
conda create -n 새환경이름 python=3.12
```

```
conda activate 새환경이름
```

```
conda install pandas matplotlib
```

파일 이름 때문에 실행이 이상할 때

Python 파일 이름을 `numpy.py`, `pandas.py`, `matplotlib.py`, `random.py`처럼 패키지 이름과 같게 만들면 문제가 생길 수 있습니다.

파일 이름은 아래처럼 구체적으로 짓습니다.

```
data_analysis_example.py
```

```
plot_example.py
```

17. 작업 습관

1. 프로젝트마다 conda 환경을 따로 만듭니다.
2. 파일과 폴더 이름은 영어, 숫자, 밑줄을 중심으로 씁니다.
3. 코드를 조금 작성한 뒤 바로 실행해 확인합니다.
4. 오류가 나면 터미널의 첫 번째 오류 메시지부터 읽습니다.
5. 논문에 들어갈 그림을 Python으로 만들 때는 원본 코드와 데이터 파일을 함께 보관합니다.
6. 코드를 어느 정도 작성하면 Git으로 커밋하고 GitHub에 올려 백업합니다.

18. 확인 체크리스트

아래 항목이 모두 되면 Python 코딩을 시작할 준비가 된 것입니다.

- ☐ Miniconda 또는 Anaconda가 설치되어 있다.
- ☐ `conda --version`이 실행된다.
- ☐ `python --version`이 실행된다.
- ☐ VS Code가 설치되어 있다.
- ☐ VS Code에 필요한 확장(Python 등)을 설치했다.
- ☐ `knue-python` conda 환경을 만들었다.
- ☐ VS Code에서 `knue-python` 환경을 인터프리터로 선택했다.
- ☐ VS Code 터미널 종류(Command Prompt/PowerShell)를 고를 수 있다.
- ☐ `hello.py`를 실행할 수 있다.
- ☐ `plot_example.py`를 실행해 그래프를 볼 수 있다.
- ☐ AI 코딩 도구(GitHub Copilot, Claude Code, GPT Codex)에 Python 코드를 요청할 수 있다.
- ☐ `ena_example.py`를 실행해 `ena_example.png`를 만들 수 있다.
- ☐ Git을 설치하고 `git --version`이 실행된다.
- ☐ GitHub 저장소에 코드를 올릴 수 있다.

19. 공식 참고 링크

- Miniconda 설치 안내: <https://www.anaconda.com/docs/getting-started/miniconda/install>
- Anaconda 설치 안내: <https://www.anaconda.com/docs/getting-started/anaconda/install>
- conda Windows 설치 안내: <https://docs.conda.io/projects/conda/en/stable/user-guide/install/windows.html>
- VS Code 다운로드: <https://code.visualstudio.com/download>
- VS Code Python 안내: <https://code.visualstudio.com/docs/languages/python>
- VS Code Python 환경 안내: <https://code.visualstudio.com/docs/python/environments>
- VS Code 확장 설치 안내: <https://code.visualstudio.com/docs/editor/extension-marketplace>
- VS Code 통합 터미널 안내: <https://code.visualstudio.com/docs/terminal/basics>
- VS Code에서 소스 제어(Git) 사용: <https://code.visualstudio.com/docs/sourcecontrol/overview>
- GitHub Copilot in VS Code: <https://code.visualstudio.com/docs/copilot/getting-started>
- Git 다운로드(Windows): <https://git-scm.com/download/win>
- Git 공식 문서: <https://git-scm.com/doc>
- GitHub 시작 안내: <https://docs.github.com/en/get-started>